PROCESSO 001

MANUAL DE DESENVOLVIMENTO E PUBLICAÇÃO DE JOGOS

Padrões Oficiais para HTML5, Unity e Godot
Integração KRUUJ SDK & Poki Standard

Da ideia à publicação: um processo para produzir rápido,
sem abrir mão da qualidade e segurança Kruuj.

Versão 1.1 | Outubro de 2026 | Kruuj Studio & Desenvolvedores Externos

1. Propósito e regra central

Este manual estabelece o processo oficial para conceber, desenvolver, testar, empacotar e publicar jogos na plataforma Kruuj. Seu objetivo é aumentar a velocidade de produção sem sacrificar estabilidade, qualidade visual, experiência mobile, segurança ou reputação da plataforma.

REGRA DE OURO: nenhum jogo entra em produção pública apenas porque "abre" ou porque o desenvolvimento parece concluído. Todo jogo deve passar pelo checklist de qualidade, funcionar nos dispositivos-alvo, ter uma versão identificável e ser aprovado pelo validador automático (check-game.js).

O processo vale inicialmente para jogos produzidos pela Kruuj Studio e será a base de adaptação futura para estúdios parceiros e Kruuj Students.

2. Escopo e princípios

- **Priorizar jogos que iniciem rapidamente**, sejam compreensíveis sem explicação longa e tenham um ciclo de jogo claro.
- **Projetar para navegador desde o início**; mobile não é uma adaptação opcional feita no fim.
- **Manter cada jogo isolado**: erros, áudio, estado ou CSS de um jogo não devem quebrar a plataforma nem outros jogos.
- **Preferir escopo pequeno e finalizado** a muitos jogos publicados com defeitos.
- **Não publicar código/build sem responsável**, versão, instruções de controle, thumbnail e testes registrados.
- **Usar bibliotecas, imagens, fontes, músicas e efeitos sonoros** com licença compatível e documentada (LICENSES.md).

3. Fluxo oficial de produção

P0 — Proposta: Registrar nome provisório, gênero, público, mecânica principal, diferencial, referência visual e estimativa de escopo. Confirmar que a ideia é suficientemente distinta e viável.

P1 — Mini-GDD e escopo: Definir core loop, condição de vitória/derrota, controles, duração típica, progressão, telas necessárias e o que fica fora da primeira versão.

P2 — Protótipo jogável: Validar a mecânica central com arte provisória. O objetivo é provar diversão e viabilidade, não polir.

P3 — Vertical slice: Produzir um trecho representativo com arte, áudio, interface, controles, carregamento e desempenho próximos do final.

P4 — Produção: Construir conteúdo restante, menus, pausa/reinício, estados de vitória/derrota, tratamento de erros e ajustes de acessibilidade.

P5 — QA interno: Executar testes funcionais, responsividade, compatibilidade, performance, áudio, segurança e regressão. Registrar bugs.

P6 — Candidato a release (RC): Congelar escopo, atribuir versão, gerar build limpa e executar checklist final em desktop e mobile.

P7 — Publicação controlada: Publicar primeiro em ambiente de teste/staging ou em acesso limitado. Confirmar telemetria básica e ausência de falhas críticas.

P8 — Release e monitoramento: Publicar para todos, registrar versão e data, monitorar erros/feedback e manter plano de rollback.

4. Documento mínimo de cada jogo

Cada jogo deve ter uma página de projeto no Notion ou documentação equivalente com os seguintes itens:

- ID único (ex.: KJ-001), nome, responsável e estado atual.
- Mini-GDD: conceito, público, core loop, controles, condições de vitória/derrota e duração esperada.
- Plataformas suportadas, orientação preferida, navegadores-alvo e requisitos mínimos.
- Lista de funcionalidades incluídas e explicitamente excluídas do MVP do jogo.
- Inventário de assets e licenças/fontes (LICENSES.md).
- Link do repositório, instruções para executar localmente e procedimento de build/exportação (README.md).
- Versão atual, changelog, bugs conhecidos e resultado do último QA.
- Thumbnail, ícone, título público, descrição curta e instruções de controle.

5. Requisitos gerais para jogos HTML5

5.1 Estrutura e inicialização

• O jogo deve abrir por uma URL de staging e funcionar por HTTPS no ambiente final.

• Entregar uma build autocontida e reproduzível; não depender de caminhos locais, arquivos do computador do desenvolvedor (file://, C:\, localhost) ou servidores temporários.

• Usar caminhos relativos para assets; verificar maiúsculas/minúsculas nos nomes dos arquivos (servidores Linux são case-sensitive).

• Não iniciar áudio automaticamente antes de uma interação do usuário. Exibir botão/ação clara para começar e habilitar áudio.

• Mostrar estado de carregamento quando houver download/preparação perceptível; evitar telas de loading que nunca terminam.

• Se algum recurso essencial falhar, apresentar mensagem recuperável em vez de tela branca.

5.2 Arquitetura e isolamento

• Todo o conteúdo do jogo deve ficar dentro do contêiner/player definido pela plataforma (iframe).

• Evitar CSS global que afete a página hospedeira; prefixar classes ou usar módulos/escopo.

• Não sobrescrever window/document handlers da plataforma sem necessidade. Remover listeners e timers ao encerrar/reiniciar cenas.

• **Não usar eval**, new Function, document.write, scripts remotos desconhecidos ou dependências sem avaliação.

• Proibido uso de diálogos nativos bloqueantes (alert, confirm, prompt) que travam o loop do navegador mobile.

• Não manipular ou redirecionar a janela hospedeira (window.parent.location ou window.top.location).

• Não armazenar dados pessoais desnecessários. Qualquer analytics deve seguir a política de privacidade da plataforma (LGPD).

5.3 Performance e Limites Numéricos

• Carregar primeiro o essencial e adiar recursos não necessários ao início.

• Comprimir imagens e áudio; usar formatos modernos (WebP, MP3 128kbps, OGG) e manter fallback quando necessário.

• Evitar loops de jogo que criem objetos, listeners ou elementos DOM a cada frame sem limpeza.

• Controlar partículas, física, animações e efeitos para dispositivos de menor desempenho.

• Usar delta time ou mecanismo equivalente para que a simulação não dependa de FPS fixo.

• Monitorar consumo de memória, travamentos e tempo de carregamento em dispositivo real.

Recurso / Métrica	Limite Recomendado	Teto Máximo Homologado
Peso Total do Pacote	Até 10 MB	18 MB (HTML5) / 25 MB (Unity/Godot)
Imagens Individuais	Até 250 KB (WebP)	500 KB
Áudios Individuais	Até 800 KB (mp3/ogg)	1.5 MB (proibido .wav sem compressão)
Tempo de Loading (C2P)	Menor que 2.5s	4.0s (em redes 4G)

5.4 Controles e UX

• Exibir controles na tela inicial ou em uma dica curta dentro do jogo.

• Oferecer teclado e mouse quando fizer sentido; jogos de toque devem ter controles touch reais, não apenas teclas simuladas.

• Botões touch devem ser grandes (mínimo 44x44px), separados e posicionados fora das áreas cobertas pelos dedos; evitar depender de hover.

• Respeitar safe areas em celulares com notch e manter botões importantes afastados das bordas.

• **Zona Livre do Botão Poki (Mobile):** No celular, o portal KRUUJ projeta um botão flutuante de saída no canto superior esquerdo (top: 14px, left: 16px). Elementos do HUD devem manter recuo seguro de **68px** desse canto.

5.5 Responsividade mobile

```
• Tag viewport obrigatória: <meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover, user-scalable=no">
```

```
• Bloqueio de rolagem acidental no CSS: * { touch-action: none; -webkit-touch-callout: none; user-select: none; }
```

• Usar layout fluido e escala proporcional; evitar dimensões fixas que cortem HUD, menus ou botões.

• Testar telas estreitas, telas altas, mudança de orientação e teclado virtual.

5.6 Áudio e acessibilidade

- Disponibilizar controle de volume e opção de silenciar quando houver áudio contínuo.
- Respeitar as restrições de autoplay dos navegadores (áudio só inicia após toque/clique voluntário).
- Não comunicar estado somente por som; usar também sinais visuais.
- Manter contraste suficiente, legibilidade e evitar flashes rápidos.

6. Padrão de pacote HTML5

A entrega deve conter, no mínimo, um pacote organizado e uma instrução de execução:

```
game-kj-001/
  index.html        # Ponto de entrada obrigatório
  README.md         # Nome/ID, versão, controles, orientação, como executar
  CHANGELOG.md      # Histórico de alterações e correções
  LICENSES.md       # Assets, dependências de terceiros e licenças
  assets/           # Pasta de imagens, áudio e fontes (se não inline)
    images/
    audio/
    fonts/
  src/              # Código-fonte (se houver processo de build)
```

7. Regras para engines: Unity e Godot

7.1 Quando usar Unity

Use Unity quando o jogo justificar o peso e a complexidade da engine: 3D, física/sistemas mais elaborados, ferramentas específicas ou uma equipe já produtiva nela. Para um microgame 2D simples, compare o custo de build, carregamento e manutenção antes de escolher.

- Exportar para Web/WebGL usando uma versão LTS aprovada pela equipe, sempre que possível.
- Configurar canvas/resolução para adaptar-se ao contêiner e evitar dimensões fixas que cortem a interface.
- Testar compressão, memória, tamanho de build e tempo de carregamento; builds Unity podem ser muito maiores que HTML5 leve.
- Validar teclado, mouse e touch em dispositivos reais; não assumir que uma interação feita para desktop funcionará no celular.
- Não depender de funcionalidades nativas que não existam no build Web.
- Entregar os arquivos completos exportados, instruções de build, versão da Unity e projeto-fonte no repositório.

7.2 Quando usar Godot

Godot é uma opção para jogos 2D e 3D quando a equipe deseja uma engine integrada e fluxo de exportação web. A escolha entre renderer, versão e recursos deve ser validada com um protótipo cedo, antes de investir no jogo inteiro.

- Exportar para Web com templates correspondentes à versão usada e documentar a versão exata do Godot.
- Testar a build em HTTPS e nos navegadores-alvo; verificar requisitos de threading/headers se o projeto depender deles.
- Preferir recursos e plugins comprovadamente compatíveis com exportação web.
- Conferir carregamento de fontes, áudio, input, resolução, fullscreen e retorno ao foco depois de trocar de aba.
- Entregar projeto-fonte, exportação completa, instruções e lista de plugins/add-ons com licenças.

7.3 Regra comum para Unity/Godot

A engine não é aprovação automática. A build exportada precisa cumprir os mesmos critérios de qualidade, responsividade, desempenho, segurança e integração de qualquer jogo HTML5.

- Não enviar apenas o executável de Windows/macOS/Linux; a entrega principal para a plataforma deve ser a versão web acordada.
- Não publicar build de desenvolvimento com console, overlays, debug menus ou logs sensíveis.
- Testar no domínio/ambiente de staging da Kruuj, pois comportamento local pode diferir do servidor.
- Se o jogo não atingir o desempenho ou o tempo de carregamento aceitável, otimizar ou reavaliar a engine/escopo.

8. Integração com a plataforma Kruuj (KRUUJ SDK)

O **KRUUJ SDK** é o canal oficial de comunicação entre o jogo e o portal. Ele é **100% retrocompatível com a API da Poki (window.PokiSDK = window.KruujSDK)**.

Inclusão no index.html: `<script src="/sdk/v1/kruuj-sdk.js"></script>`

Ciclo de Vida Obrigatório em Código:

```
1. Inicialização e Fim de Loading: KruujSDK.init().then(() => KruujSDK.gameLoadingFinished());
2. Início de Partida Ativa (clique em Jogar): KruujSDK.gameplayStart();
3. Pausa, Game Over ou Retorno ao Menu: KruujSDK.gameplayStop();
4. (Opcional) Comerciais / Rewarded Ads: KruujSDK.commercialBreak(); ou KruujSDK.rewardedBreak();
```

9. QA: checklist obrigatório

9.1 Funcional

- [ ] O jogo abre e chega ao menu/início sem erro.
- [ ] Todos os controles principais funcionam.
- [ ] Regras, pontuação, vitória/derrota e reinício funcionam.
- [ ] Não há softlocks, telas vazias ou caminhos sem saída.
- [ ] Pause/resume e mudança de aba não quebram o jogo.
- [ ] Links e botões fazem o que prometem.

9.2 Desktop

- [ ] Testado em navegador Chromium (Chrome/Edge/Opera).
- [ ] Testado em Firefox ou outro segundo navegador.
- [ ] Resolução pequena e grande verificadas.
- [ ] Teclado, mouse, fullscreen e áudio verificados.

9.3 Mobile

- [ ] Testado em pelo menos um aparelho Android real e iPhone/iPad real.
- [ ] Orientação e rotação comportam-se corretamente.
- [ ] Botões e controles podem ser usados sem toques acidentais.
- [ ] HUD, menus e telas finais não ficam cortados.
- [ ] Não há scroll/zoom involuntário durante a jogabilidade.
- [ ] Performance e aquecimento/bateria são aceitáveis para a duração prevista.

9.4 Qualidade e release

- [ ] Sem bugs críticos ou altos abertos.
- [ ] Sem assets ausentes, erros repetidos no console ou áudio quebrado.
- [ ] Tempo de carregamento e tamanho da build registrados e dentro dos limites.
- [ ] Thumbnail, ícone, título, descrição e controles estão prontos.
- [ ] Licenças de assets e dependências foram verificadas.
- [ ] Versão, changelog, build e responsável estão registrados.
- [ ] Build final foi testada no staging da Kruuj e validada pelo check-game.js.

10. Severidade de bugs e bloqueio de publicação

Severidade	Exemplos	Regra de Bloqueio
S0 — Crítico	Não abre, trava, crash, vazamento de janela, risco de segurança, eval()	Bloqueia release; corrigir imediatamente
S1 — Alto	Progressão impossível, controles essenciais falham, mobile inutilizável, zoom bloqueado	Bloqueia release
S2 — Médio	Bug contornável, UI parcialmente ruim, áudio secundário falha, evento de SDK ausente	Corrigir antes do release, salvo exceção
S3 — Baixo	Detalhe visual, texto menor, melhoria não essencial, README ausente	Pode entrar no backlog com prazo

Qualquer exceção a S1/S2 deve ser aprovada pelo responsável do produto e documentada com impacto e workaround.

11. Versionamento, publicação e rollback

- Usar versionamento semântico: MAJOR.MINOR.PATCH (ex.: 1.2.0).
- Manter changelog com mudanças, correções e problemas conhecidos.
- Não sobrescrever silenciosamente uma build publicada sem registrar a versão.
- Guardar a última build estável para rollback imediato.
- Após publicar, fazer smoke test no site real: abrir, iniciar, jogar, pausar/reiniciar e sair.

12. Publicação: ficha de submissão

[ ] ID e nome do jogo definidos.
[ ] Responsável e contato interno definidos.
[ ] Build web completa anexada ou vinculada.
[ ] Link do repositório e instruções de build disponíveis.
[ ] Thumbnail/ícone no padrão da plataforma.
[ ] Descrição curta e controles revisados.
[ ] Plataformas e orientação declaradas.
[ ] Checklist QA preenchido com data e testador.
[ ] Lista de bugs conhecidos aprovada.
[ ] Direitos e licenças verificados.
[ ] Versão e changelog preenchidos.
[ ] Aprovação final registrada via validador automático (check-game.js).

13. Responsabilidades e aprovação

Papel	Responsabilidade
Game Developer	Implementar, manter build, corrigir bugs, documentar e executar testes locais com o validador.
QA / Testador	Executar checklist independente, registrar passos para reproduzir e classificar severidade.
Responsável pelo jogo	Confirmar escopo, conteúdo, qualidade final e materiais de publicação.
Responsável pela plataforma	Validar integração, comportamento no player/staging, conformidade com SDK e segurança.
Release owner	Autorizar publicação, registrar versão e garantir rollback disponível.

14. Métricas de qualidade

Começar com poucas métricas e registrar dados por jogo:

- Taxa de sucesso de carregamento.
- Tempo de carregamento percebido.
- Erros/crashes reportados.
- Percentual de sessões mobile sem falhas.
- Taxa de conclusão ou duração da sessão.
- Feedback dos jogadores e bugs por versão.

15. Processo para parceiros e Kruuj Students

- Enviar o Kruuj Game Standard antes de iniciar a integração.
- Solicitar build web, fonte/projeto conforme o acordo, instruções e lista de dependências.
- Exigir declaração de autoria e comprovação de direitos sobre assets e música.
- Não prometer publicação automática: toda submissão passa por avaliação técnica e editorial.
- Definir por contrato propriedade intelectual, licença de distribuição, remuneração e suporte.
- Estudantes devem ter autorização para publicar trabalhos acadêmicos e assets de terceiros.

16. Template de relatório de QA

```
JOGO / ID:
VERSÃO:             BUILD / URL:
DATA:               TESTADO POR:
DISPOSITIVOS / NAVEGADORES:
RESULTADO: [ ] APROVADO    [ ] REPROVADO    [ ] APROVADO COM PENDÊNCIAS
[ ] Funcional       [ ] Desktop      [ ] Mobile real      [ ] Performance/Áudio
BUGS ENCONTRADOS:
- ID / Severidade:
- Passos para reproduzir:
- Resultado esperado vs observado:
DECISÃO FINAL / RESPONSÁVEL:
```

17. Definition of Done — quando um jogo está pronto

[ ] A experiência principal está completa e sem bugs S0/S1.

[ ] Os requisitos S2 foram corrigidos ou têm exceção formal aprovada.

[ ] O jogo obtém o status APROVADO no validador check-game.js.

[ ] A build web funciona no staging da Kruuj.

[ ] Desktop e mobile foram testados conforme o público-alvo.

[ ] Carregamento, responsividade, controles, áudio e telas finais foram verificados.

[ ] README, changelog, versão e links técnicos estão atualizados.

[ ] Assets e dependências têm licença compatível.

[ ] Thumbnail, descrição e controles estão prontos.

[ ] QA foi registrado e a aprovação final foi dada.

[ ] Existe uma build estável anterior ou plano de rollback.

18. Controle do documento

Campo	Valor
Documento	KRUUJ-PROC-001 — Manual de Desenvolvimento e Publicação de Jogos
Versão	1.1
Status	Processo oficial consolidado com Kruuj SDK e Validação Automatizada
Responsável	Kruuj Studio / Liderança do Projeto
Aplicação	Jogos HTML5 e jogos exportados para Web por Unity/Godot
Revisão sugerida	Após os primeiros 3 jogos publicados e depois a cada mudança no pipeline

19. Anexo Técnico: Como Executar o Validador de Jogos

A plataforma KRUUJ disponibiliza a ferramenta automatizada check-game.js para que qualquer estúdio parceiro ou desenvolvedor interno valide seu jogo em segundos.

Desenvolvedor Externo (Execução Local via CMD):

```
node check-game.js ./pasta-do-seu-jogo
```

Desenvolvedor Interno (Monorepo KRUUJ):

```
npm run check:game <slug-do-jogo>
npm run check:games # Valida todos os jogos do catálogo
```

O validador checa: integridade de index.html, caminhos absolutos, isolamento, bloqueio de eval/alert, viewport mobile blindado, touch-action: none, autoplay de áudio, chamadas do Kruuj SDK (loadingFinished, gameplayStart, gameplayStop) e limites de peso.